

Introduccion a Ruby

Juan Ignacio Raggio

May 19, 2025

Contents

1	Diferencias con Java	2
1.1	Vamos a usar Ruby 3.4	2
2	Hello world	2
3	No existen los operadores	2
4	Constantes	2
5	Comillas simples " vs comillas dobles "	2
6	Strings: Metodos	3
7	Variables	3
7.1	Variables de clase	3
7.2	Variables de instancia	3
8	Metodos	3
8.1	Metodos de instancia	3
8.2	Metodos de clase + self.class vs ClassName	4
9	Herencia Ruby	4
9.1	Herencia de metodos privados	4
10	Metodos	4
10.1	Metodos privados	4
10.1.1	Organizacion de privados vs publicos	5
10.2	Metodos Protegidos	5
10.3	Metodos con argumentos variables	5

11 Poner el return si es en la ultima linea es de mal estilo	5
12 to_s (toString en Ruby)	5
13 Inspect	6
14 Clases abstractas en Ruby	7

1 Diferencias con Java

- Es interpretado
- Tipado dinamico
- Todo es un objeto
- Garbage collector
- Manejo de excepciones
- Programacion funcional

1.1 Vamos a usar Ruby 3.4

2 Hello world

```
puts "Hello world"
```

3 No existen los operadores

- El <, >, == son todos metodos de instancia de cada objeto

4 Constantes

- Se escriben en mayusculas, si se cambia tira warning

5 Comillas simples " vs comillas dobles “”

- Las comillas simples son para tomar literal lo que esta dentro de las comillas

- Las comillas dobles son para tener **formato** -> Si no se quiere formato es ineficiente usar comillas dobles

```
name = 'juani'
saludo = "Hola #{name}"
```

6 Strings: Metodos

```
'1234'.size # 4
'1234ñ'.ascii_only:? # false
'abc'.capitalize
```

7 Variables

7.1 Variables de clase

- Comienzan con @@

7.2 Variables de instancia

- Comienzan con @
- No se pueden tener dos metodos con el mismo nombre (no existe el polimorfismo parametrico)
- Las variables de instancia son todas **privadas** si quiero permitir acceso hay que hacer un getter

```
class Date
  def initialize(day, mont, year)
    # variables de instancia
    @day = day
    @month = month
    @year = year
  end
end
```

8 Metodos

8.1 Metodos de instancia

- Comienzan con def

- Son todos publicos

```
# Dos formas de escribir lo mismo
def hello_world
  puts 'Hello world'
end
```

```
def simple_method = puts 'Hi!'
```

8.2 Metodos de clase + self.class vs ClassName

```
class Father
  def self.default_make = 'Father'

  def father_one = self.class.default_make
  def father_two = Father.default_make
end
```

9 Herencia Ruby

- Se usa el operador < que es equivalente al extends de Java

```
class Son < Father
  def self.default_make = 'Son'
end
```

9.1 Herencia de metodos privados

- Las clases hijas pueden acceder a los metodos privados de la clase padre

10 Metodos

10.1 Metodos privados

- Un metodo privado solo puede ser accedido por la propia instancia

```
class Father
  private def aux_method # mal estilo
    # private code
  end
```

```

    def some_method(other) = other.aux_method
end

# Falla
father = Father.new
father.aux_method

# Falla tambien
a = Father.new
b = Father.new
b.some_method(a) # Pues a no es this dentro de b entonces no tiene acceso

```

10.1.1 Organizacion de privados vs publicos

- Hay dos opciones estan en la diapo

10.2 Metodos Protegidos

- A diferencia de los metodos privados, otras clases si pueden acceder a los metodos protegidos
- Pero siguen siendo “privados” en cuanto al main u otra clase externa

10.3 Metodos con argumentos variables

- Un argumento que puede tener un conjunto de valores
- Solo se puede tener un argumento variable

```

def foo(a, b, *v, c)

end

```

11 Poner el return si es en la ultima linea es de mal estilo

12 to_s (toString en Ruby)

```

class Person
  def initialize(first_name, last_name)

```

```

        @first_name = first_name
        @last_name = last_name
    end
end

person = Person.new('John', 'Doe')
puts person

class Person
  def to_s
    "Person #{@first_name} #{@last_name}"
  end
end

puts person

```

13 Inspect

```

class Person
  def initialize(first_name, last_name)
    @first_name = first_name
    @last_name = last_name
  end
end

person = Person.new('John', 'Doe')

# En vez de hacer to_s usa inspect
p person

class Person
  def inspect
    "Person #{@first_name} #{@last_name}"
  end
end

p person

```

14 Clases abstractas en Ruby

```
class AbstractClass
  def initialize
    raise 'Cannot instantiate an abstract class'
  end

  def init(some_var)
    @some_var = some_var
  end

  private :init
end
```